NOTE The examples for this section can be found in the `questionwithpayload` package if no package is explicit in the snippet.

When you specify the `createRequest` parameter for the request, you can't rely on `Worker.Parse.apply()` as you did before. Remember that `createRequest` expects a function with only one parameter so the system can pass the `replyTo` for you:

```
package akka.actor.typed.scaladsl

trait ActorContext ... {

  ...

  def ask[Req, Res](target: RecipientRef[Req], createRequest: ActorRef[Res]
  ➥=> Req)(...)
  ...
}
```

But if `Worker.Parse.apply` has two parameters, `text` and `replyTo`, you can't just pass it to `createRequest`. You need to create a function that fills in the `text` parameter and leaves the `replyTo` parameter unfilled. Then you can pass this function to `createRequest`. You can do this with a technique called *currying*.

Currying allows you to pass some parameters to a function and leave others empty. The result of currying is a function with fewer parameters than the original function. For example, if you have the function `multiplication(x: Int)(y: Int) = x * y`, you can curry it like `multiplication(4)`: that is, you pass only the `x`, not the `y`. This way, you get a new function, `multiplicationCurried(z: Int) = 4 * z`, and you